Lecture: Cyclomatic Complexity

What is Cyclomatic Complexity?

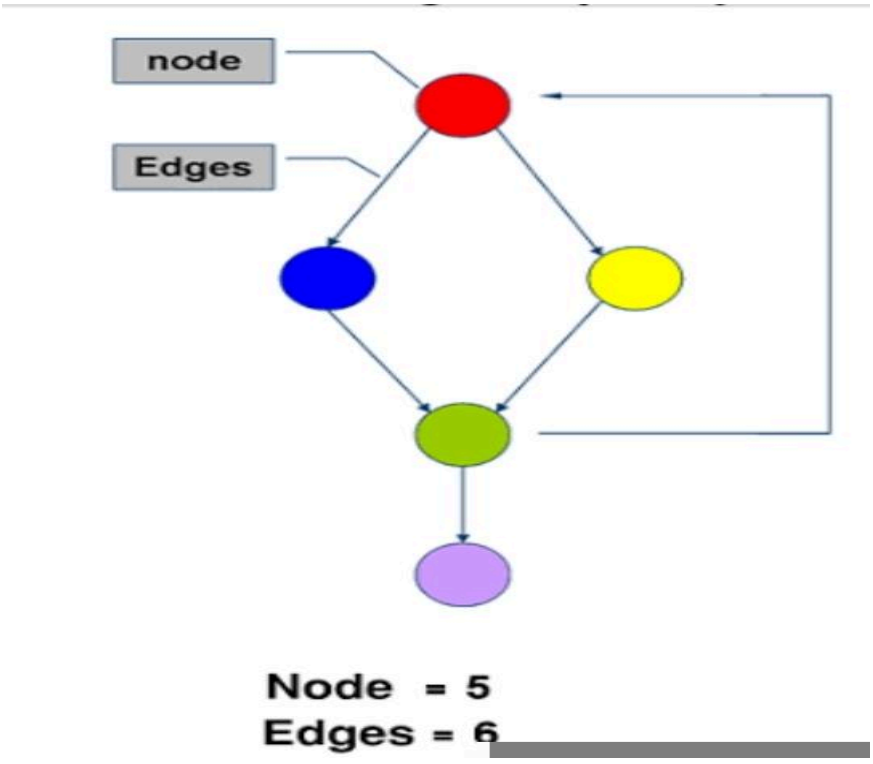
Cyclomatic complexity, developed by **Thomas J. McCabe, Sr.** in 1976, is a software metric that provides a **quantitative measure of the number of linearly independent paths** through a program's source code.

---

2. The Control Flow Graph (CFG)

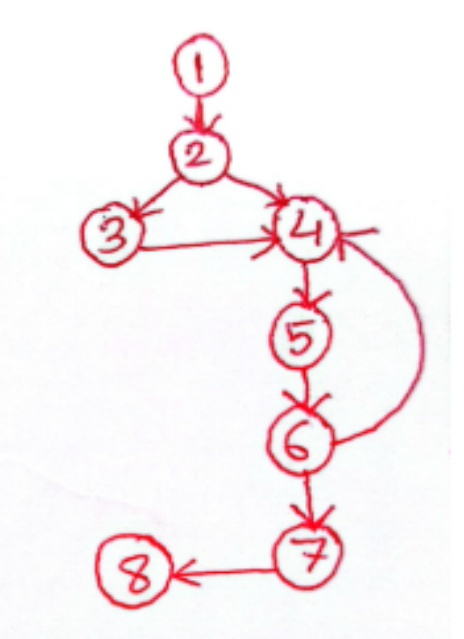
To calculate cyclomatic complexity, we first represent the program's logic using a **Control Flow Graph (CFG)**.

- **Nodes (N)):** Represent one or more sequential statements that are executed without any branching (known as a "basic block").
- **Edges (E):** Represent the flow of control from one node to another. These are the arrows in the graph.
- **Predicate Nodes (P):** Nodes that contain a condition (like if, while, for) and have more than one outgoing edge, representing a decision point.



3. Creating CFG from Adjacency Matrix

	1	2	3	4	5	6	7	8
1	0	1	0	0	0	0	0	0
2	0	0	1	1	0	0	0	0
3	0	0	0	1	0	0	0	0
4	0	0	0	0	1	0	0	0
5	0	0	0	0	0	1	0	0
6	0	0	0	1	0	0	1	0
7	0	0	0	0	0	0	0	1
8	0	0	0	0	0	0	0	0



4. Formulas for Calculation

Cyclomatic complexity, denoted as V(G) , can be calculated using one of three primary methods:

Method 1: Using Edges, Nodes, and Connected Components

This is the most general formula, based on graph theory:

V(G) = E - N + 2P

Where:

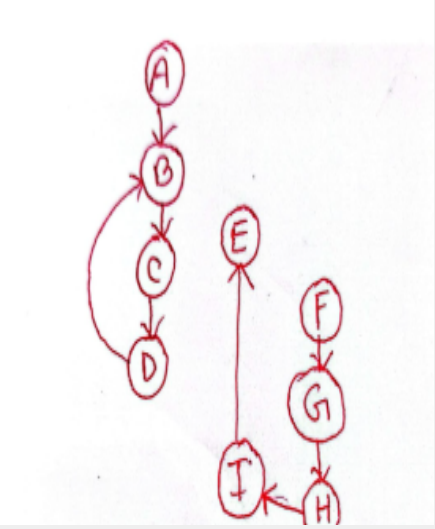
- E = Number of **Edges** (control paths)
- N = Number of **Nodes** (basic blocks)
- P = Number of **Connected Components** in the graph (P=1) for a single program, function, or method).

For a single program (P=1), the formula simplifies to:
V(G) = E - N + 2

Example Solved :

	A	B	C	D	E	F	G	H	I
A	0	1	0	0	0	0	0	0	0
B	0	0	1	0	0	0	0	0	0
C	0	0	0	1	0	0	0	0	0
D	0	1	0	0	0	0	0	0	0
E	0	0	0	0	0	**0**	**0**	**0**	**0**
F	**0**	**0**	**0**	**0**	**0**	0	1	0	0
G	**0**	**0**	**0**	**0**	**0**	0	0	1	0
H	**0**	**0**	**0**	**0**	**0**	0	0	0	1
I	**0**	**0**	0	**0**	**1**	0	0	0	0

CFG



Following the CFG from Adjuncency Matrix:

- Number of **Edges** (control paths) E = 8
- Number of **Nodes** (basic blocks) N= 9 (A, B, C, D, E, F, G, H, I)
- Number of **Connected Components** in the graph P = 2 (two disconnected Components)

For a single program (P=1)), the formula simplifies to:
V(G) = E - N + 2 P
= 8- 9 + 2 x 2
= 3
The cyclomatic Complexity is 3
Which means

- **Simple** procedure, well-structured.
- Low risk. Generally acceptable complexity.

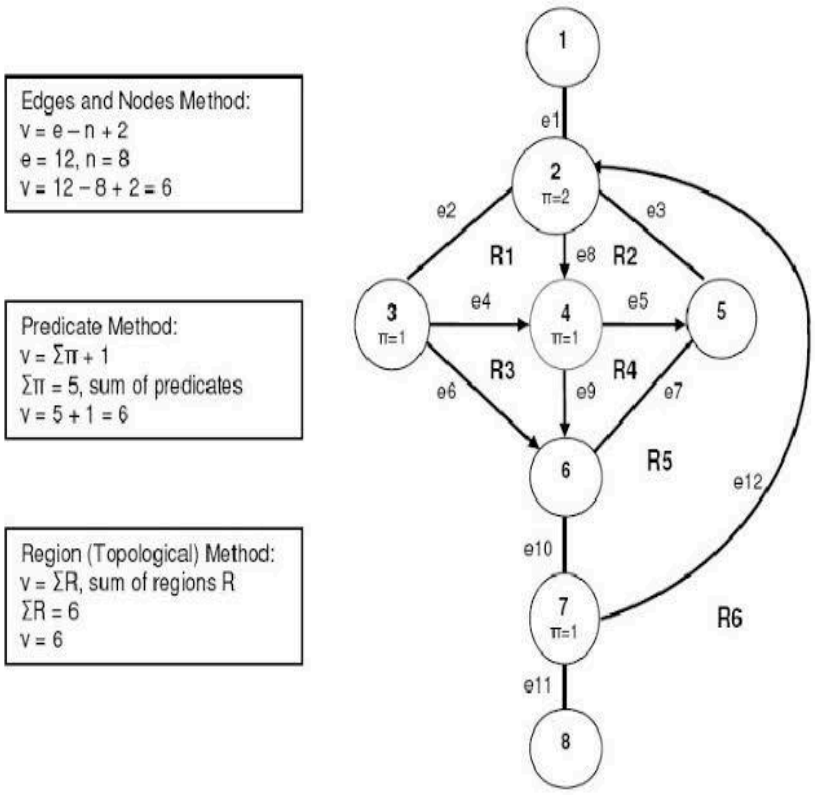
Method 2: Using Predicate Nodes (Decision Points)

This is the most practical method for a single component:

V(G) = P+ 1

Where:

- P = Number of **Predicate Nodes** (or decision points like if, while, for, case in a switch, and conditional operators like && or ||).



Number of **Predicate Nodes P = 5 (2, 2, 3, 4, 7)**

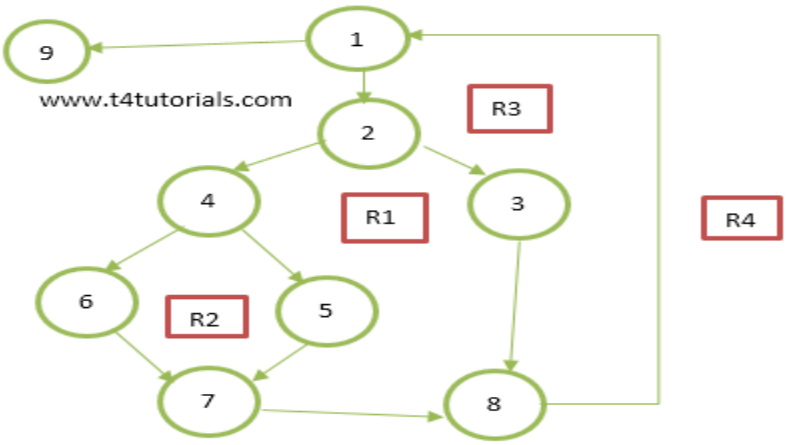
Method 3: Using Regions

If the graph is drawn on a plane, the complexity is equal to the number of regions created by the graph.

V(G) = R

Where:

- R= Number of **Closed Regions** in the graph, **plus the exterior region**.



In this CFG Number of **Closed Regions is 3 (R1, R2, R3)**

And **the exterior region is 1 (R4)**

So, here R = 4

v(G) = R = 4

4. Application and Interpretation

The complexity score is a critical indicator in software engineering and testing:

V(G) Score Range	Interpretation	Risk & Action
1-10	**Simple** procedure, well-structured.	Low risk. Generally acceptable complexity.
11-20	**More Complex**, requires careful testing.	Moderate risk. Candidates for refactoring.
21-50	**Complex**, high risk of defects.	High risk. **Strong candidates for immediate refactoring.**
> 50	**Untestable Code**.	Very high risk. Code must be simplified.

Key Uses:

1. **Test Case Determination:** The V(G) value provides the **minimum number of test cases** required to ensure every independent path in the code is tested, achieving 100\% branch coverage (Basis Path Testing).
2. **Code Quality:** It identifies modules that are too complex, often violating the Single Responsibility Principle, and are prone to errors and difficult to maintain.
3. **Risk Management:** High-complexity modules are flagged as high-risk areas, allowing development and testing teams to prioritize their efforts

Exercises on Disconnected CFG:

1. From the following Adjacency Matrix outlining the CFG, Calculate the Cyclomatic Complexity by a suitable method.

	a	b	c	d	e	f	g	h
a	0	1	0	0	0	0	0	0
b	0	0	1	1	0	0	0	0
c	1	0	0	1	0	0	0	0
d	0	0	0	0	1	0	0	0
e	0	0	0	0	0	0	0	0
f	0	0	0	0	0	0	1	0
g	0	0	0	0	0	0	0	1
h	0	0	0	0	0	0	1	0

2. From the following Adjacency Matrix outlining the CFG, Calculate the Cyclomatic Complexity by a suitable method.

	1	2	3	4	5	6	7	8	9
1	0	1	0	0	0	0	0	0	0
2	0	0	1	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0
4	0	0	0	0	1	0	0	0	0
5	0	0	1	0	0	0	0	0	0
6	0	0	0	0	0	0	1	0	1
7	0	0	0	0	0	0	0	1	0
8	0	0	0	0	0	0	0	0	1
9	0	0	0	0	0	0	0	0	0

Exercises for CFG with Connected Graph (where connected component p = 1):

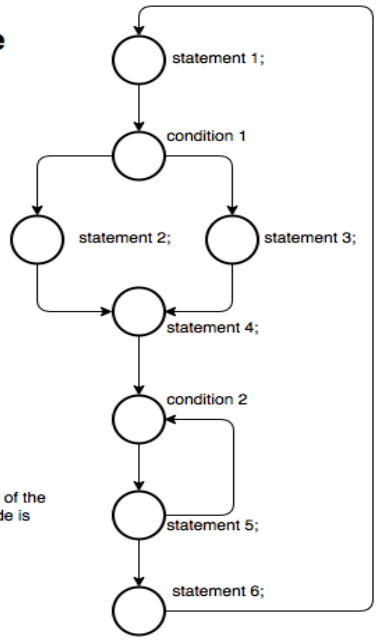
3. From the following Adjacency Matrix outlining the CFG, Calculate the Cyclomatic Complexity by a suitable method.

	a	b	c	d	e	f	g	h
a	0	**1**	0	0	0	0	0	0
b	0	0	1	**1**	0	0	0	0
c	0	0	0	0	**1**	0	0	0
d	0	0	0	0	0	1	0	0
e	0	0	0	0	0	0	1	0
f	0	0	0	0	0	0	0	1
g	0	0	0	1	0	0	0	0
h	0	0	0	0	0	1	0	0

Solved Problem:

Sample code

```
statement 1;
If ( condition 1 ) {
        statement 2;
} else {
        statement 3;
}
statement 4;
for( condition 2 ) {
        statement 5;
}
statement 6;
```



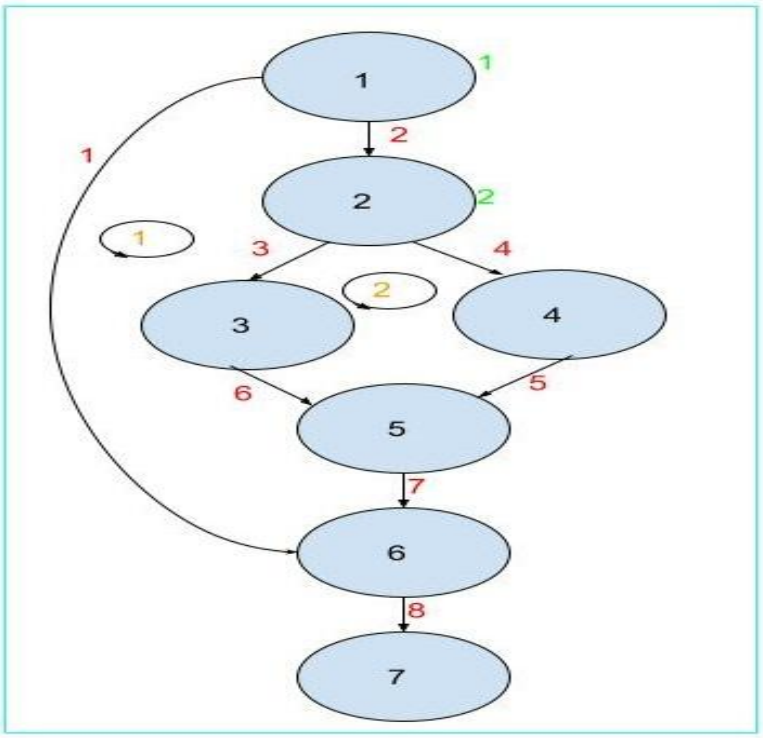
Complexity

The cyclomatic complexity of the graph representing the code is

v(G) = E - N + 2

= 9 - 8 + 2

= 3




```
V(G) = E - N + 2 * P
 = 8 - 7 + 2 * 1 = 3
```

In the above code and its control flow graph, there are two conditional statements, IF X = 300, and THEN IF Y > Z, hence there are two conditional nodes(P) namely, 1, and 2, denoted in green in the control flow graph. So as per the formula,

```
V(G) = P + 1
 = 2 + 1 = 3
```

In the above control flow graph, there are two closed regions(R), denoted by two transparent circles namely 1, and 2 denoted in orange in the control flow graph. So as per the formula,

```
V(G) = R + 1
 = 2 + 1 = 3
```